MASTER'S MODULE · AGENTIC AI · WINTER SEMESTER

Reasoning Models & Agent Design

What changes when the model thinks for itself — and what that costs.

Unit 04 · Part 2 of 2 · Session 05 · Week 5

Prof. Dr. Hauke Schramm · Department of Computer Science · Kiel University

Today's agenda

1. **Reasoning models** trained or distilled to spend more compute at inference
2. **Inference-time scaling** test-time compute as a conditional third dial
3. **Consequences for agents** CoT redundancy, planning moves inward
4. **Economics & hybrid designs** thinking-token costs, model routing
5. **Benchmarks & skepticism** reading capability claims critically

Where we are

Unit 04, part 2 — from prompting techniques to model capabilities

Last week (S04) — we engineered reasoning by hand: CoT prompts, scaffolds, format control

This week — models trained to reason internally — what survives of that craft?

The shift — from scripting how the model thinks to deciding when thinking pays

Next unit — orchestration builds on both: who plans, who executes, who verifies

GUIDING QUESTION

When the model brings its own reasoning, what is left for the agent designer to decide?

1

What reasoning models are

Trained or distilled to spend more compute at inference

Reasoning models, defined

Definition — an LLM trained or distilled to allocate extra inference computation to multi-step problem solving

Thinking tokens — generated and billed like output; whether the trace is shown depends on the implementation

Learned, not prompted — the reasoning behaviour comes from training, not your instructions

Reward signal (RL path) — verifiable outcomes — unit tests pass, the math answer matches

Distilled path — supervised fine-tuning on reasoning demonstrations; no RL of its own

CONTRAST WITH S04

Last week we elicited reasoning with prompts. A reasoning model brings its own, whichever way it was trained.

Three training paths to a reasoning model

As published with DeepSeek-R1 (DeepSeek-AI, 2025, sections 2.2 to 2.4)

R1-Zero: pure RL

Base model, then RL on verifiable rewards with no supervised reasoning data. Self-checks and backtracking emerge; readability and language mixing suffer.

R1: multi-stage

Cold-start SFT on curated long traces, then reasoning RL, then rejection sampling plus SFT, then a further RL stage for general use.

Distillation

Supervised fine-tuning of a smaller model on traces from a strong reasoner, no RL of its own. Lab 05's deepseek-r1:7b is such a distilled model.

The lineage, 2024 to 2026

Three public milestones, then convergence

OpenAI o-series

o1 (late 2024): first widely deployed RL-trained reasoner; hidden thoughts, effort levels from Dec 2024. Folded into GPT-5 (Aug 2025) behind a router.

DeepSeek-R1

Early 2025: open weights and an open recipe — R1-Zero (pure RL), R1 (multi-stage) and six distilled small models.

Claude extended thinking

2025: hybrid design — one model, thinking switched on per request (token budget then, effort level now).

The 2026 norm

Major model families are hybrid: thinking depth is a request parameter, not a separate product.

What shows up in the thinking trace

Behaviours that emerge from outcome-based RL

Self-verification — the model re-derives and checks intermediate results unprompted

Backtracking — it abandons a failing approach mid-trace and restarts

Decomposition — it splits the problem into subgoals without a scaffold

R1-Zero finding — in R1-Zero this emerged from RL alone; R1 added cold-start data for readability

Faithfulness caveat — the visible trace is not a reliable account of the computation (Chen et al., 2025)

2

Inference-time scaling

Compute at answer time as a third dial, conditional on task and strategy

Spending test-time compute: the serial dial

Conditional gain — more test-time compute can improve accuracy; allocation and task difficulty set the gain (Snell et al., 2024)

One parameter — an effort level (OpenAI, Anthropic) or a thinking budget (Gemini), set per request

Reported curves — on hard math and code, accuracy rose roughly log-linearly with thinking tokens (OpenAI, 2024)

Task-dependent — gains reported on math, code, planning; near zero on recall and formatting; sometimes negative

Diminishing returns — budgets beyond task difficulty buy verbosity, not correctness

LAB 05, PART E

Run the local reasoning model on the deep tasks at four num_predict budgets, plot accuracy versus budget and read off the knee (R2).

Two ways to buy accuracy at inference

Serial depth inside the model vs parallel breadth outside it

Serial: think longer

- one long chain, revising and backtracking
- handles dependent steps that build on each other
- no selector needed; external validation may still be
- latency grows with the budget

Parallel: sample more

- N independent answers, then select
- majority vote — self-consistency (Wang et al., 2023)
- or best-of-N with a verifier
- latency about max sample time plus selection, if capacity allows
- N-fold cost; needs checkable or votable answers

3

Consequences for agent design

What changes when the model plans for itself

CoT prompting becomes partially redundant

Redundancy — “think step by step” adds little when the model already thinks

Interference — rigid reasoning scaffolds can fight the trained reasoning policy

Prompt shift — specify goals, constraints, output contract and required domain procedures

Planning moves inward — plan templates matter less; tune planning structure and effort empirically

CoT still earns its keep — on standard models and for domain procedures — not as an audit trail

NOT AN AUDIT TRAIL

Audit observable actions, evidence, approvals and state changes. Reasoning summaries are supplementary evidence, not a faithful execution trace (Chen et al., 2025).

Prompting the two model classes

Same task, different starting points — tune both empirically

Standard model

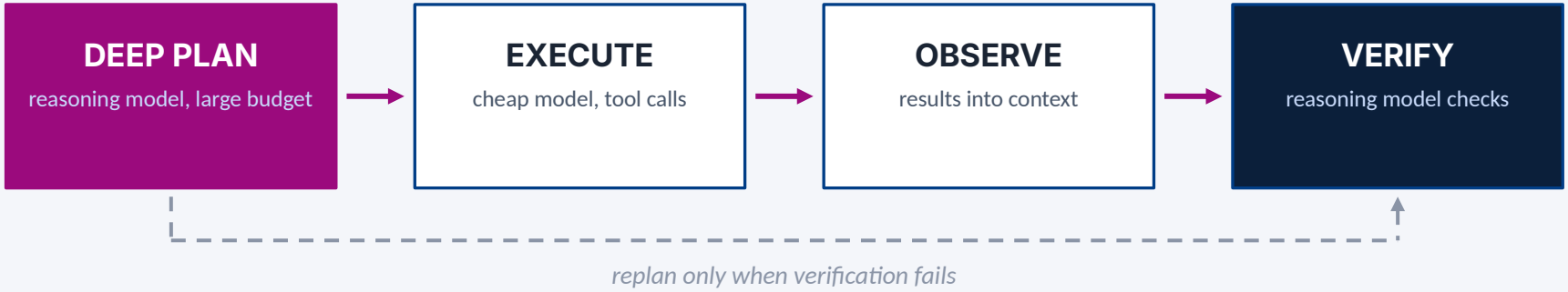
- few-shot exemplars with worked reasoning
- explicit plan template to fill in
- “show your work before answering”
- procedure spelled out step by step

Reasoning model

- goal, constraints, success criteria
- output contract plus required domain procedures
- thinking effort or budget set via the API
- examples to pin the format, more if the domain needs

The agent loop, reshaped

A design hypothesis: fewer, heavier iterations instead of many cheap probes



- Many-cheap loops probe and recover; fewer-heavier loops front-load the thinking.
- An architecture choice, not an automatic effect of the model class: measure quality, cost and latency against a strong single-model baseline.

A minimal model router

Routing by step kind — the starting hypothesis Lab 05 extends with a thinking-budget cap

router.py

```
PLAN = "reasoning-xl"    # slow, expensive
WORK = "standard-mini"   # fast, cheap

def model_for(step):
    if step.kind in ("plan", "verify"):
        return PLAN
    if step.retries > 0:   # escalate
        return PLAN
    return WORK

for step in agent.next_steps():
    out = llm(model_for(step), step.prompt)
    agent.observe(step, out)
```

Cheap default — most steps are routine

Escalate — planning, verification, retries

Hypothesis — beat a single-model baseline

- model choice is config, not architecture

4

Economics and hybrid designs

Paying for thought only where it pays back

Thinking-token inflation: the arithmetic

Illustrative numbers — recompute with your provider's prices

Scenario — 20-step tool-using agent run, ~400 visible output tokens each

Standard model — about 8,000 output tokens per run

Reasoning model on every step — +3,000 thinking tokens each — about 68,000 billed tokens

With a 4x price — output cost grows roughly 34-fold for the same visible text

Latency — 3,000 extra sequential tokens at 80 tok/s adds ~38 s per step

RULE OF THUMB

Pay for thinking where errors compound or are expensive to undo; often unnecessary for simple formatting. Lab 05 (R4) redoes this arithmetic with your measured numbers.

Standard vs reasoning model, side by side

Axis	Standard model	Reasoning model
Output price per token	baseline	same rate on hybrid models; thinking billed as output
Time to first answer	seconds	tens of seconds to minutes
Math, code, planning	weak to fair	stronger; often improves with budget
Extraction, formatting	strong and cheap	usually little gain; thinking tokens wasted
Steering style	needs CoT scaffolds	lean goals; scaffolds can interfere
Typical failure	shallow first guess	overthinking trivial steps

Neither column dominates — the axes tell you which step gets which model.

Hybrid design patterns

Combining cheap and reasoning models in one agent — each a hypothesis to test against one strong model

Model routing

A rule or classifier picks the model per step: cheap by default, heavy for hard steps.

Plan-execute split

Reasoning model writes the plan; cheap model executes steps; planner re-engaged on failure.

Verifier at the end

Cheap model drafts; reasoning model audits claims and repairs the weakest parts.

Escalation ladder

Retry a failed step on a stronger model before declaring the run stuck.

Choosing models per role

Generic agent roles — a starting assignment to verify per task, not a fixed mapping

Planner — reasoning model candidate — early errors compound

Tool caller — cheap model — query rewriting, tool arguments, called often

Evaluator — mid-tier — judgment needed, but volume is high

Formatter — cheap model — schema and table output, rarely needs thought

Verifier — reasoning model candidate — final audit against evidence

LAB 05, R1

You decide planner versus formatter from your own accuracy, token and latency numbers, then defend the choice against a single-model baseline.

Lab 05 this week: measure the dials yourself

Local Ollama (qwen2.5:7b vs deepseek-r1:7b), no API keys, gap-fill notebook with fold-out solutions, 90 to 120 min

Part	What you do	Deliverable
A, B	connect to Ollama; load 8 offline tasks (5 deep, 3 shallow)	chat_with_stats helper
C	parse the reasoning model's thinking trace; faithfulness check	trace reading
D	both models on all tasks: accuracy, tokens, latency	R1: planner vs formatter
E	deep tasks at four num_predict budgets; strict grading	R2: knee of the curve, plot
F	rebuild the router; add a per-run thinking-budget cap	R3 code; R4 cost arithmetic
G	self-consistency on the standard model; temperature; policies	optional exploration

Reasoning-model calls take tens of seconds locally — Parts D and E dominate the lab time.

5

Benchmarks and skepticism

Reading capability claims like a scientist

Why benchmark deltas mislead

Contamination — test items can leak into training corpora and inflate scores

Evidence — GSM1k: some models scored lower on fresh matched items, others barely (Zhang et al., 2024)

Alternatives — a drop on fresh tasks may also mean harder items or distribution shift — match difficulty

Saturation — near ceiling, small deltas can still matter but need error bars and failure types

Benchmarketing — tuned harnesses, best-of-N reported as if single-shot, cherry-picked baselines

What to do — evaluate on your own task traces: recent, private, held out

CLAIM HYGIENE

Always ask: which harness, how many samples, what thinking budget, which baseline, which uncertainty?

TAKEAWAYS

What you should remember

- **Reasoning models —** trained or distilled to spend extra inference compute: R1-Zero, R1, distillation
- **Third dial —** more test-time compute can help; task and allocation strategy set the gain
- **Prompting shifts —** goals, constraints, contract, domain procedures; tune effort empirically
- **Routing —** cheap default, reasoning model for plan and verify — a hypothesis to beat a baseline
- **Audit actions —** audit observable actions and state changes; reasoning traces are supplementary evidence
- **Skepticism —** benchmark deltas need harness details, matched difficulty and error bars

NEXT SESSION Orchestration — ReAct, plan-and-execute, DAG-based vs dynamic control.

Reading & references

- DeepSeek-AI (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948; peer-reviewed as Guo et al., Nature 645, 633-638 (2025).
  Sections 2.2 to 2.4: the R1-Zero, R1 and distillation recipes.
- OpenAI (2024). Learning to Reason with LLMs. Technical blog introducing o1.
  First public inference-time scaling curves.
- Snell, C., Lee, J., Xu, K., Kumar, A. (2024). Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters. ICLR 2025; arXiv:2408.03314.
- Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903.
  The S04 anchor — contrast prompted with trained reasoning.
- Wang, X. et al. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023; arXiv:2203.11171.
  Parallel test-time compute via majority voting.
- Chen, Y. et al. (Anthropic, 2025). Reasoning Models Don't Always Say What They Think. arXiv:2505.05410.
  Faithfulness of visible thinking traces.
- Zhang, H. et al. (2024). A Careful Examination of LLM Performance on Grade School Arithmetic (GSM1k). NeurIPS 2024 Datasets & Benchmarks; arXiv:2405.00332.
  Fresh matched items: some models drop markedly, others barely.